Exercise M3 – Building Bert-Based QnA System

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract* — This study explores the performance of four pretrained question-answering models, BERT-base-cased-SQuAD2, DistilBERT, BERT-Large, and RoBERTa, on responses to ten specific contextual questions developed from a local news article. The analysis included Exact Match, F1 score, confidence, and inference time to quantify performance according to accuracy and computational efficiency. The DistilBERT model demonstrated the best balance of performance, with the highest Exact Match and F1 scores, rapid inference speed, and consistent confidence. RoBERTa delivered comparable accuracy, while inference was slightly slower, and BERT-Large demonstrated the lowest Exact Match scores and longest inference, despite its larger sized model. The baseline BERT model provided moderate performance with which to compare the other three models. Results indicate model architecture, pre-training strategy, and fine-tuning are important considerations when evaluating both predicted performance and operational efficiency within the context of extractive question answering.**

Keywords— Question Answering System, Pretrained Models, BERT, DistilBERT, RoBERTa

I. Introduction

Question and answering computer systems have integrated into Natural Language Processing (NLP) and have made it possible for machines to increasingly understand and produce answers to human queries. The emergence of pretrained language models was a major advancement in the field that allowed for solid solutions across a variety of NLP tasks such as Q&A, sentiment analysis, and machine translation.

Models that have been pretrained, such as BERT, DistilBERT, BERT-Large, and RoBERTa, are pretrained on large collections and fine-tuned to adapt the learned representation to a specific task. Transfer learning enables the models to generalize from big data while using a relatively smaller number of additional training examples to adapt to the specialized domain. The use of pretrained models in machine learning provides a number of advantages that have been extensively documented in the literature, including reduced training time, improved performance on downstream tasks, and improved generalization. More precisely, the pretrained models can learn to represent complex language patterns and contextual information to yield more accurate and reproducible Q&A systems [1].

This study assesses and compare four pretrained Q&A models, specifically, BERT-base-cased-SQuAD2, DistilBERT, BERT-Large, and RoBERTa, on a set of domain-specific questions. The evaluation measures included Exact Match (EM), F1 score, confidence score, and inference time per query. In addition, the researcher aims to leverage these measures to evaluate and compare the trade-off between model size, speed, and accuracy such that researchers can utilize this information in their practical use cases [2].

II. Methodology

Four pretrained question-answering (QA) models, specifically, BERT-base-cased-SQuAD2, DistilBERT, BERT-Large, and RoBERTa, were examined using a domain-specific news article regarding overseas Filipino workers living in Beirut. Ten targeted questions, each with a gold-standard answer, were used to facilitate a measure of performance. The models were implemented using Hugging Face Transformers pipeline ("question-answering"), and both interactive and batch evaluations were automated using Python scripts. Exact Match, F1 score, confidence score, and inference time per query were the four metrics collected that provided a systematic approach for comparing model accuracy and efficiency.

a) News Article Used

The article titled “Chartered flights eyed for repatriation of over 200 OFWs in Lebanon” by Ruth Abbey Gita-Carlos tackled about how the government of the Philippines is presently organizing chartered flights to return home 200 plus overseas Filipino workers (OFWs) located in Beirut, Lebanon, amid cancellations of previously scheduled flights due to recent bombings. At this time, there are 111 OFWs at temporary shelters, and an additional 110 are in the process of obtaining exit permits. There are complications trying to get landing rights for chartered flights and options are now being considered for land and sea routes. Each repatriated OFW will receive PHP 150,000 in sack assistance and psychosocial services. As of now no Filipinos have been reported injured in the ongoing conflict. [3]

The context article used for this experiment was sourced from the Philippine News Agency [3], which reports on the repatriation of over 200 Overseas Filipino Workers (OFWs) from Lebanon. All ten evaluation questions were derived directly from the content of this article to assess the models’ ability to extract accurate answers.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

b) *Baseline Model*

For the baseline model provided, the code imported the libraries needed to implement a question-and-answering pipeline. The Hugging Face Transformers library, BertTokenizer and BertForQuestionAnswering, in addition to the built-in Python textwrap module for formatting were all imported. The code then loaded the baseline model, deepset/bert-base-cased-squad2 using from_pretrained(), which will load the model and tokenizer with weights that were previously fine-tuned on the SQuAD 2.0 dataset and now the model is capable of accurately extracting answers from text by locating the answers as the most relevant spans in context.

```
from transformers import pipeline, BertTokenizer, BertForQuestionAnswering
import textwrap

#Load the fine-tuned model
model = BertForQuestionAnswering.from_pretrained('deepset/bert-base-cased-squad2')
tokenizer = BertTokenizer.from_pretrained('deepset/bert-base-cased-squad2')
```

Fig. 1. BERT Baseline Model Initialization

Afterwards, the code will create a Hugging Face pipeline by combining the baseline model and tokenizer into a single object. This abstraction makes the workflow easier, as the user don't have to handle tokenization, inference, and output formatting ourselves, and allows for user-friendly interactions to have the model perform extractive question-answering. Here, the user will input a context article, which the code processes with textwrap.dedent() and .strip() to remove extra whitespace and formatting discrepancies. Afterwards, the code print the article wrapped through the textwrap.fill(), which improves readability and facilitates queried interaction.

```
#Create a pipeline for question answering
qna_pipeline = pipeline('question-answering', model=model, tokenizer=tokenizer)

context = input("enter Context Article: ")
dedented_text = textwrap.dedent(context).strip()
print("Context Article:\n")
print(textwrap.fill(dedented_text, width=120))
```

Fig. 2. QnA Pipeline and Context Article Input

The final section is an interactive loop that enables the user to continually ask questions about the article. Each question is processed by the pipeline to compare with the context text and return the answer prediction, character span, and confidence score. The loop will continue, even if the user poses multiple questions, until the user types the character *, allowing users to explore the text without boundaries iteratively. This builds the baseline performance for the BERT model, which will be measured against subsequent pretrained models.

```
inquiry = input("\nType your question: ")
while (inquiry!='*'):
  answer = qna_pipeline({'question':inquiry, 'context': context})
  print("Answer found: "+answer ['answer'])
  print("At Index: ", answer ['start']," - ",answer ['end'])
  print("with probability:", answer['score'],"\n")
  inquiry = input("enter another question(* to stop):")
```

Fig. 3. Question Input and BERT Answer Output with Probability Score

c) *Baseline Model Evaluation Block*

Using the baseline model configuration, an evaluation framework was constructed manually for assessing model performance quantitatively. This evaluation code was not integrated into the original interactive pipeline, but was included in order to systematically evaluate metrics across the questions chosen for analysis. The evaluation environment imports Python's time package to time inference latency and pandas to organize the results into a more readable table format.

```
import time
import pandas as pd

# Simple F1 function for token overlap
def compute_f1(pred, gold):
    pred_tokens = pred.lower().split()
    gold_tokens = gold.lower().split()
    common = set(pred_tokens) & set(gold_tokens)

    if not common:
        return 0.0

    precision = len(common) / len(pred_tokens)
    recall = len(common) / len(gold_tokens)
    f1 = 2 * (precision * recall) / (precision + recall)
    return round(f1, 3)
```

Fig. 4. Baseline Evaluation Initialization and F1 Formula Setup

A helper function, called compute_f1, obtains token-level F1 scores when comparing predicted answers to the gold-standard answers. This function takes both strings, transforms them to lowercase, tokenizes each of them, and calculates the overlap between the predicted tokens and reference tokens. The F1 score is a versatile metric that accounts for partial correctness when the model's prediction is semantically correct but stated differently than the gold answer. Exact Match (EM) was calculated as well so that we could have a stricter measure of correctness in addition to the F1 score.

The evaluation set is defined by a specific form of ten questions from the context article and its associated gold-standard answers. These questions create an evaluation structure for the model's ability to extract relevant information accurately and consistently.

```python
# Define questions + gold answers
questions = [
    "How many overseas Filipino workers are set for repatriation from Beirut?",
    "Who announced the plan for chartered flights for OFWs?",
    "Why were the scheduled flights of around 15 OFWs on Sept. 25 cancelled?",
    "How many OFWs are currently staying in temporary shelters in Beirut?",
    "How many OFWs are applying for exit permits from the Lebanese government?",
    "What challenges does the Philippine government face in arranging chartered flights?",
    "What alternative routes are being considered aside from air travel?",
    "How much financial assistance will each repatriated OFW receive?",
    "Which militant group was targeted by Israel's intensified airstrikes?",
    "Were any Filipinos hurt in the recent attacks?"
]

gold_answers = [
    "more than 200",          # Q1
    "Bernard Olalia",         # Q2
    "recent bombings",        # Q3
    "111",                    # Q4
    "110",                    # Q5
    "securing landing rights",# Q6
    "land and sea routes",    # Q7
    "PHP150,000",             # Q8
    "Hezbollah",              # Q9
    "No"                      # Q10
]
```

Fig. 5. 10 Listed Questions Used and Gold Standard Answers

Specifically, during evaluation, the baseline BERT model predicts answers to each of the questions, and the pipeline records the predictions, along with their confidence score, and the inference time. At the end of the evaluation, EM and F1 are estimated for each question, and collate all into a pandas DataFrame. In addition, a summary row is added to display averages for EM, F1, confidence, and inference time across all questions so the row averages are easy to compare and comprehend. From this evaluation approach, the user can systematically benchmark the performance of the baseline model and compare with other additional pretrained models.

```python
# Collect results
results = []
for q, gold in zip(questions, gold_answers):
    start = time.time()
    answer = qna_pipeline({'question': q, 'context': context})
    end = time.time()

    predicted = answer['answer']
    inference_time = end - start
    EM = 1 if predicted.strip().lower() == gold.strip().lower() else 0
    F1 = compute_f1(predicted, gold)

    results.append({
        "Question": q,
        "Gold Answer": gold,
        "Predicted Answer": predicted,
        "Exact Match": EM,
        "F1": F1,
        "Confidence": round(answer['score'], 3),
        "Inference Time (s)": round(inference_time, 3)
    })

df = pd.DataFrame(results)

summary = {
    "Question": "AVERAGE",
    "Gold Answer": "-",
    "Predicted Answer": "-",
    "Exact Match": df["Exact Match"].mean(),
    "F1": df["F1"].mean(),
    "Confidence": df["Confidence"].mean(),
    "Inference Time (s)": df["Inference Time (s)"].mean()
}
df = pd.concat([df, pd.DataFrame([summary])], ignore_index=True)

print("\nEvaluation Results:\n", df.to_string(index=False))
```

Fig. 6. Prediction Results Collection and Summary Table

The deepset/bert-base-cased-squad2 model as the baseline mode has a number of advantages that makes it a good reference point for extractive question answering tasks. The model was trained with SQuAD 2.0, and achieves quite high accuracy on producing exact answers by having strong Exact Match and F1 scores, with the score model estimating its probability of correctly answering both answerable and unanswerable questions [4].

It can also be used in common question answering frameworks like Haystack and Hugging Face's Transformers library allowing for easy DIY and deployment into different use cases which can include routines like document search, chatbots, and automated customer support [4].

As a BERT based model, deepset/bert-base-cased-squad2 uses bidirectionality to create context by attending to the pattern of dependencies of words in both directions [4]. The various features of deepset/bert-base-cased-squad2 contribute to it being a feasible enough baseline for evaluating performance of question answering systems.

d) Pretrained BERT Models Selected

I.) DistilBERT Model

DistilBERT is a lighter version of the Bidirectional Encoder Representations from Transformers (BERT) model that was released by Hugging Face to offer a more efficient option for natural language processing tasks. Through knowledge distillation, DistilBERT retains approximately 97% of BERT's language understanding and is 40% smaller and 60% faster than BERT, making it an attractive option for tasks with limited computation or time constraints [5].

The distillation process consists of training a smaller version, the "student", and hoping that it learns the behavior of a larger version, the "teacher", by learning from how the teacher distributed its output. This is useful because it allows DistilBERT to learn from the "dark knowledge," or nuanced information, contained in the teacher's predictions and ultimately maintains many of the original model's performance from the teacher. DistilBERT stripped down the architecture, so the token-type embeddings and pooler layer the used for next-sentence prediction, leading to an even smaller version with little sacrifice to performance, as it would have been for the original model. [5]

Due to its performance and efficiency, DistilBERT is especially valuable in resource-limited settings, including mobile and deployment contexts. Its capability to generate meaningful answers with relatively low processing requirements ensures it is a helpful resource when creating natural language processing applications that need to scale and be agile. [5]

II.) BERT-Large Model

BERT-Large, created by Google AI, is a more advanced version of the original BERT architecture to one that better models more complex language patterns. BERT-Large consists of 24 transformer layers, each with 16

attention heads, with a hidden size of 1024, with a total of 340 million parameters. BERT-Large, because of its increased capacity, is able to better model more complicated linguistic structures and dependencies than the BERT-Base model. The model is also "uncased," or does not distinguish between uppercase and lowercase letters, which can be useful for some natural language processing tasks.[6]

One distinctive aspect of BERT-Large is that during pretraining, it is pretrained with Whole Word Masking (WWM). BERT-Large learns representations of words more coherently with WWM because it masks an entire word by masking all the subword tokens of that word at once. Practically, this means WWM masks entire words, a stark contrast to traditional masked language models that mask individual subwords. It is advantageous for the model to not simply rely on portion of a word. In general, WWM formed BERT-Large's pretraining positive because it outperformed the similar-sized masked language models at a variety of natural language understanding tasks including, for instance, question answering. After pre-training, BERT-Large, similar to other pre-trained, task agnostic models, is fine-tuned for specific tasks, such as in the settings of the Stanford Question Answering Dataset (SQuAD) to adapt its general understanding of language to the particulars of task. [6]

The combination of large model size and the WWM technique make BERT-Large especially suited for tasks requiring deep contextual understanding and precision in information retrieval. BERT-Large has a higher computational demand, so, it is best suited for environments where one has adequate resources to support specifications. Despite these considerations, BERT-Large performance has shifted the state of the art in natural language processing by demonstrating the advantages of scaling up model size and pretraining. [6]

III.) RoBERTa Model

RoBERTa, designed by Facebook AI, is an improved and modified version of the BERT model, intended to improve performance by changing various elements of the pretraining process. While BERT uses a fixed mask pattern during pretraining, allowing the same tokens to be masked in the same way across all pretraining epochs, RoBERTa uses dynamic masking, so that every single token gets masked differently across pretraining epochs, allowing the model to learn a greater variety of language representations. In addition, RoBERTa is trained with larger mini-batches and on more data to improve generalization. All these changes lead to better performance on a variety of natural language understanding tasks, such as sentiment analysis and question answering. [7]

The deepset/roberta-base-squad2 model is a version of RoBERTa that was fine-tuned on the SQuAD 2.0 dataset, which includes answerable and unanswerable questions, so it can extract precise answers from a context, or note when a question is unanswerable based on the given context. It shows strong performance with an F1 score of 82.91% and an exact match score of 79.87% on the SQuAD 2.0 dev set. These results confirm its usefulness on difficult question-answering tasks, which may include unanswerable questions. The transformer architecture in combination with the libraries that are possible to use with Haystack make it an efficient and practical model for creating question-answering systems. [7]

e) DistilBERT, BERT-Large, and RoBERTa Code Model

The interactive Q&A (Question-Answering) structure for DistilBERT, BERT-Large, and RoBERTa retain the same architecture as the baseline BERT model so that the modeling comparisons remain consistent. Initially, the environment is set up, and the essential Python libraries are imported, which include time used to track inference speed, pandas used to store and organize results in a table, and textwrap is used to format the input text for readability. There is also a Hugging Face pipeline object instantiated for each model, in which the appropriate pretrained model weights are then loaded. For instance,

- DistilBERT has the weights loaded using distilbert/distilbert-base-cased-distilled-squad,
- BERT-Large loads the weights called bert-large-uncased-whole-word-masking-finetuned-squad, and
- RoBERTa loads the weights using roberta-base-squad2.

```
import time
import textwrap
import pandas as pd
from transformers import pipeline

model_name = "distilbert/distilbert-base-cased-distilled-squad"
qna_pipeline = pipeline("question-answering", model=model_name)

# Input context article
context = input("Enter Context Article: ")
dedented_text = textwrap.dedent(context).strip()
print("\nContext Article:\n")
print(textwrap.fill(dedented_text, width=120))
```

Fig. 7. Pretrained Models Initialization and Context Article Input

Each pipeline caries out the processes of tokenization, extracting answers, and scoring confidence automatically, facilitating question-answering tasks downstream. The user is prompted to enter an article of context where there is cleaning/dedentation and then outputted to the console for readability. The displayed article is used as the written basis for every query.

The compute_f1 helper function is used to compute the token-level F1 score between predicted answers and gold standard answers. The function converts both answers to lower-case, tokenizes them, and computes the overlap, providing a partial credit score. This f-score is helpful when using extractive question answering since the semantically correct answer may not match with the reference answer

word-for-word, so the f1 score is a more lenient metric than strict exact match (EM).

```python
def compute_f1(pred, gold):
    pred_tokens = pred.lower().split()
    gold_tokens = gold.lower().split()
    common = set(pred_tokens) & set(gold_tokens)

    if not common:
        return 0.0

    precision = len(common) / len(pred_tokens)
    recall = len(common) / len(gold_tokens)
    f1 = 2 * (precision * recall) / (precision + recall)
    return round(f1, 3)
```

Fig. 8. F1 Formula Setup

A total of ten pre-defined questions alongside gold standard answers are then defined. The questions were selected to cover important pieces of information present in the context, including numeric information, proper nouns, events, and policy information. When running the evaluation, users may optionally provide a single question to test the model interactively. For each question (including the user defined and the ten pre-defined questions) the pipeline predicts an answer in the text, records the answer's character span in the text, evaluates the confidence score, and collects inference time.

```python
questions = [
    "How many overseas Filipino workers are set for repatriation from Beirut?",
    "Who announced the plan for chartered flights for OFWs?",
    "Why were the scheduled flights of around 15 OFWs on Sept. 25 cancelled?",
    "How many OFWs are currently staying in temporary shelters in Beirut?",
    "How many OFWs are applying for exit permits from the Lebanese government?",
    "What challenges does the Philippine government face in arranging chartered flights?",
    "What alternative routes are being considered aside from air travel?",
    "How much financial assistance will each repatriated OFW receive?",
    "Which militant group was targeted by Israel's intensified airstrikes?",
    "Were any Filipinos hurt in the recent attacks?"
]

gold_answers = [
    "more than 200",
    "Bernard Olalia",
    "recent bombings",
    "111",
    "110",
    "securing landing rights",
    "land and sea routes",
    "PHP150,000",
    "Hezbollah",
    "No"
]
```

Fig. 9. Listed Questions and Gold-Standard Answers

```python
custom_question = input("\nType your question (or '*' to skip): ")
if custom_question != '*':
    answer = qna_pipeline({'question': custom_question, 'context': context})
    print(f"\nAnswer found: {answer['answer']}")
    print(f"At Index: {answer['start']} - {answer['end']}")
    print(f"Confidence: {round(answer['score'],3)}\n")

# Evaluation on 10 questions
results = []
for q, gold in zip(questions, gold_answers):
    start = time.time()
    answer = qna_pipeline({'question': q, 'context': context})
    end = time.time()

    predicted = answer['answer']
    inference_time = end - start
    EM = 1 if predicted.strip().lower() == gold.strip().lower() else 0
    F1 = compute_f1(predicted, gold)

    results.append({
        "Question": q,
        "Gold Answer": gold,
        "Predicted Answer": predicted,
        "Exact Match": EM,
        "F1": F1,
        "Confidence": round(answer['score'],3),
        "Inference Time (s)": round(inference_time,3)
    })
```

Fig. 10. Question Input and Answer Output and Evaluation

The results are compiled in a formatted pandas DataFrame which includes the predicted answers, exact match and F1 score values, confidence score, and inference times. A summary row also calculates the averages across all questions, offering an overview of the model's performance. This consistent evaluation framework allows for a fair comparison of DistilBERT, BERT-Large, and RoBERTa models as well as allowing for quantitative evaluation and a qualitative examination of the model's behavior in extracting accurate answers from context passages.

```python
df = pd.DataFrame(results)
summary = {
    "Question": "AVERAGE",
    "Gold Answer": "-",
    "Predicted Answer": "-",
    "Exact Match": df["Exact Match"].mean(),
    "F1": df["F1"].mean(),
    "Confidence": df["Confidence"].mean(),
    "Inference Time (s)": df["Inference Time (s)"].mean()
}
df = pd.concat([df, pd.DataFrame([summary])], ignore_index=True)

print("\nEvaluation Results:\n", df.to_string(index=False))
```

Fig. 11. Evaluation Table Code

f) Visualizations

The code begins by creating a pandas DataFrame that includes average, minimum, and maximum values for Exact Match, F1 score, confidence, and inference time. The formatting of the DataFrame allows for direct comparisons to be made easily. The next segment of code creates a plotting function to show the metrics in line charts, where the average is shown as a red line and the min-max range is represented as a shaded area below the red line. In addition, data labels for the averages are included for clarity. This approach, when visualized effectively, provides a quick snapshot of the model's performance and

variability across all metric, while enabling the researchers to view them all within a single workflow.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np


comparison_data = {
    "Model": [
        "BERT-base-cased-SQuAD2 (baseline)",
        "DistilBERT",
        "BERT-Large",
        "RoBERTa"
    ],
    "Exact Match Avg": [0.5, 0.8, 0.4, 0.7],
    "F1 Avg": [0.6933, 0.8844, 0.7001, 0.8734],
    "F1 Max": [1.0, 1.0, 1.0, 1.0],
    "F1 Min": [0.0, 0.368, 0.0, 0.226],
    "Confidence Avg": [0.5563, 0.7103, 0.5456, 0.5613],
    "Confidence Max": [0.978, 1.148, 0.963, 1.261],
    "Confidence Min": [0.007, 0.368, 0.177, 0.135],
    "Inference Time Avg (s)": [4.5531, 1.4598, 9.9985, 2.8008],
    "Inference Time Max (s)": [5.246, 2.157, 10.294, 10.294],
    "Inference Time Min (s)": [4.091, 1.333, 9.265, 2.558]
}

comparison_df = pd.DataFrame(comparison_data)

print("Comparison Table:")
display(comparison_df)
```

Fig. 12. Initialization and Comparison Table Code

```python
sns.set(style="whitegrid")

def plot_line_with_range(df, avg_col, min_col, max_col, title, ylabel):
    x = df['Model']
    y_avg = df[avg_col]
    y_min = df[min_col]
    y_max = df[max_col]

    plt.figure(figsize=(10,6))
    plt.plot(x, y_avg, marker='o', color='red', label='Average')
    plt.fill_between(x, y_min, y_max, color='skyblue', alpha=0.3, label='Min-Max Range')

    for i, val in enumerate(y_avg):
        plt.text(i, val+0.02*max(y_max), f"{val:.2f}", ha='center', va='bottom', fontsize=9)

    plt.title(title)
    plt.ylabel(ylabel)
    plt.xticks(rotation=25, ha='right')
    plt.ylim(0, max(y_max)*1.15)
    plt.legend()
    plt.tight_layout()
    plt.show()

plot_line_with_range(comparison_df, "Exact Match Avg", "Exact Match Avg", "Exact Match Avg",
                     "Exact Match per Model", "Exact Match Avg")

plot_line_with_range(comparison_df, "F1 Avg", "F1 Min", "F1 Max",
                     "F1 Score per Model", "F1 Score")

plot_line_with_range(comparison_df, "Confidence Avg", "Confidence Min", "Confidence Max",
                     "Confidence per Model", "Confidence")


plot_line_with_range(comparison_df, "Inference Time Avg (s)", "Inference Time Min (s)", "Inference Time Max (s)",
                     "Inference Time per Model", "Inference Time (s)")
```

Fig. 13. Line Graphs for Each Metrics Code

III. Results

The performance of four pre-trained QA models, BERT-base-cased-SQuAD2, DistilBERT, BERT-Large, and RoBERTa was evaluated on a set of ten context-specific questions based on a Philippine News Agency article about the repatriation of Filipino workers. Evaluation metrics included Exact Match (EM) and F1 score, confidence, and inference time. All results and raw outputs are fully recorded in the accompanying Excel and notebook files to provide data for our analyses as well as graphical representations discussed in the Discussion section.

All models were evaluated on the identical set of ten questions to maximize consistency of testing, and to aid comparison. The questions were constructed to encapsulate key factual information from the passage and included questions that asked about specific numeric data, named entities, and events from the passage. Gold standard answers were provided to compute EM and F1 scores in regards to the model predictions. The questions and gold answers used for predictive evaluation follow:

Questions:

1. How many overseas Filipino workers are set for repatriation from Beirut?
2. Who announced the plan for chartered flights for OFWs?
3. Why were the scheduled flights of around 15 OFWs on Sept. 25 cancelled?
4. How many OFWs are currently staying in temporary shelters in Beirut?
5. How many OFWs are applying for exit permits from the Lebanese government?
6. What challenges does the Philippine government face in arranging chartered flights?
7. What alternative routes are being considered aside from air travel?
8. How much financial assistance will each repatriated OFW receive?
9. Which militant group was targeted by Israel's intensified airstrikes?
10. Were any Filipinos hurt in the recent attacks?

Gold-Standard Answers:

1. More than 200
2. Bernard Olalia
3. Recent bombings
4. 111
5. 110
6. Securing landing rights
7. Land and sea routes
8. PHP150,000
9. Hezbollah
10. No

a) Baseline Model

```
Type your question: Who announced the plan for chartered flights for OFWs?
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/question_answe
  warnings.warn(
Answer found: Bernard Olalia
At Index:  428  -  442
with probability: 0.16282296180725098

enter another question(* to stop):*
```

Fig. 14. Baseline Model Input Demonstration

```
Evaluation Results:
                                   Question                  Gold Answer                                                  Predicted Answer  Exact Match    F1  Confidence  Inference Time (s)
          How many overseas Filipino workers are set for repatriation from Beirut?             more than 200                                       200          0.0 0.5000      0.4000              5.0000
                   Who announced the plan for chartered flights for OFWs?                Bernard Olalia                                   Bernard Olalia          1.0 1.0000      0.3330              4.0910
          Why were the scheduled flights of around 15 OFWs on Sept. 25 cancelled?               recent bombings                 the recent bombings in Beirut.          0.0 0.5714      0.5670              6.1230
                   How many OFWs are currently staying in temporary shelters in Beirut?                   111                                           111          1.0 1.0000      0.5260              5.2000
                 How many OFWs are applying for exit permits from the Lebanese government?                   110                                           110          1.0 1.0000      0.5260              4.2000
 What challenges does the Philippine government face in arranging chartered flights?   Securing landing rights  Philippine government is facing several challenges, including securing landing rights.          0.0 0.4000      0.5000              5.1000
                 What alternative routes are being considered aside from air travel?              land and sea routes                                 land and sea routes          1.0 1.0000      0.5000              5.1000
                          How much financial assistance will each repatriated OFW receive?                PHP150,000                                      PHP150,000          1.0 1.0000      0.5000              4.0910
                   Which militant group was targeted by Israel's intensified airstrikes?                 Hezbollah                                       Hezbollah.          0.0 0.8000      0.5700              4.0910
                                   Were any Filipinos hurt in the recent attacks?                        No                                        no Filipinos were hurt          0.0 0.4000      0.5200              5.2530
                                                                                                                                                                 -          0.5 0.6933      0.5563              4.5531
```

Fig. 15. Raw Evaluation Results of Baseline Model

The baseline BERT model was assessed against ten context-driven questions with Exact Match (EM), F1, confidence, and inference time as the measures of assessment. The average EM score was 0.50 and F1 was 0.693. The average EM score indicates that half of the answers were precisely matched to the gold standard and that the majority of predictions still had significant word-

token overlap with the correct answers. The average confidence scores varied from near-zero to nearly one, indicating varying levels of model certainty for the correct or partially correct predictions. The average inference time per question on CPU was approximately 4.55 seconds, indicating a moderate level of computational efficiency on an extractive Q&A task.

b) DistilBERT Model

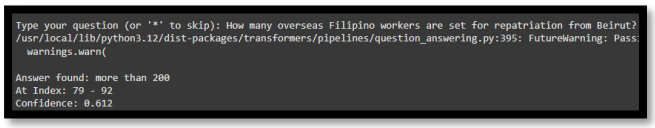


Fig. 16. DistilBERT Input Demonstration

Fig. 17. DistilBERT Raw Evaluation Resutls

The DistilBERT model was assessed using the same ten context-specific questions as the baseline model. DistilBERT produced an average Exact Match (EM) of 0.80 and an average F1 score of 0.884, indicating that most predictions were fully or largely correct. Inconsistencies in phrasing for some of the answers, for example,

- "because of the recent bombings in Beirut" for "recent bombings" and,
- "no Filipinos were hurt" for "No",

kept them from being perfect EM scores, but they did contribute to higher F1 values. The confidence scores ranged from 0.368 to 0.979 demonstrating the model generally had a strong confidence level for each answer. The average inference time per question was 1.46 seconds on CPU, which illustrates DistilBERT's efficiency relative to the baseline model when providing high accuracy on extractive Q&A tasks.

c) BERT-Large Model

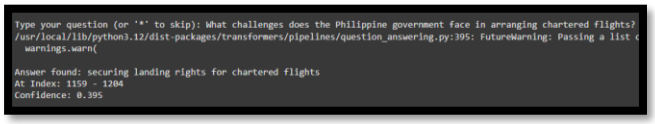


Fig. 18. BERT-Large Input Demonstration

Fig. 19. BERT-Large Raw Evaluation Results

BERT-Large was assessed using the same ten context-specific questions as the other models. The average Exact Match (EM) score was .40 and the F1 score was .700. This indicates that the predicted answers were often different in wording from the gold standard, but there was typically a reasonable amount of token-level overlap. For example,

- "DMW Undersecretary Bernard Olalia" instead of "Bernard Olalia" and
- "the sea and the land route" instead of "land and sea routes"

would lower EM but keep the proper meaning. The confidence scores ranged from .301 to .963, which was good to see as some answers were much more certain than others. The average inference time per question was approximately ten seconds on CPU, which was much longer than DistilBERT because of the size and complexity of the model. These results show that BERT-Large captures semantic information, even if penalties for strict EM led to a reduction in its measured accuracy.

d) RoBERTa Model

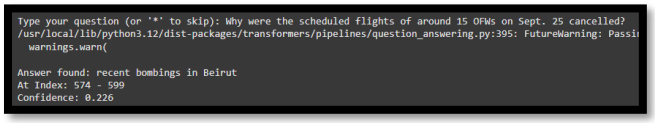
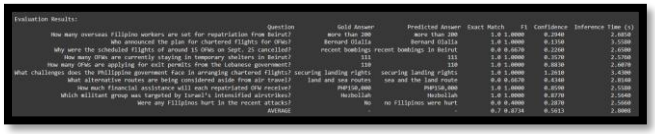


Fig.20. RoBERTa Input Demonstration



Fig, 21. RoBERTa Raw Evaluation Results

RoBERTa underwent testing on the same ten context-specific questions that all other models mentioned above were tested on. RoBERTa scored an average Exact Match (EM) score of 0.70 and an average F1 score of 0.873, suggesting that most sentences with predicted answers matched the gold standard or were token matches but with minor word differences in things like

- "recent bombings in Beirut" versus "recent bombings" or
- "sea and the land route" versus "land and sea routes."

These minor wording differences prevented a perfect EM score with the correct answer, but the meaning remained correct. The confidence scores made by RoBERTa varied widely from 0.287 to 1.261, meaning that numeric confidence did not truly correlate to correctness. The average inference time per question was 2.8 seconds on CPU, meaning RoBERTa had a nice balance of speed and accuracy compared to other larger models like BERT-Large.

e) Comparison Table

	Model	Exact Match Avg	F1 Avg	F1 Max	F1 Min	Confidence Avg	Confidence Max	Confidence Min	Inference Time Avg (s)	Inference Time Max (s)	Inference Time Min (s)
0	BERT-base-cased-SQuAD2 (baseline)	0.5	0.6933	1.0	0.000	0.5563	0.978	0.007	4.5531	5.246	4.091
1	DistilBERT	0.8	0.8844	1.0	0.368	0.7103	1.148	0.368	1.4598	2.157	1.333
2	BERT-Large	0.4	0.7001	1.0	0.000	0.5456	0.963	0.177	9.9985	10.294	9.265
3	RoBERTa	0.7	0.8734	1.0	0.226	0.5613	1.261	0.135	2.8008	10.294	2.558

Fig. 22. Raw Comparison Table

Metrics	Baseline Model	DistilBERT	BERT-Large	RoBERTa
Exact Match Score Average	0.5	0.8	0.4	0.7
F1 Score Average	0.6933	0.8844	0.7001	0.8734
F1 Score Max	1.0	1.0	1.0	1.0
F1 Score Minimum	0.000	0.4	0.000	0.4
Confidence Score Average	0.5563	0.7103	0.5456	0.5613
Confidence Score Max	0.978	1.148	.963	1.261
Confidence Score Minimum	0.007	0.368	0.177	0.135
Inference Time Average	4.5531	1.4598	9.9985	2.8008
Inference Time Max	5.246	2.157	10.294	3.430
Inference Time Minimum	4.091	1.333	9.265	2.558

Table 1. Comparison Table

DistilBERT obtained an overall Exact Match of 0.80, with an F1 score of 0.884, suggesting that it performed well while still being fast, with an average inferencing time of 1.46 seconds per question. RoBERTa's results were close behind DistilBERT at Exact Match of 0.70, F1 of 0.873, and a good trade-off of speed at 2.8 seconds per question. The baseline BERT's results were in the moderate range at Exact Match of 0.50, F1 of 0.693, and expand upon the general trend with an inferencing time of 4.55 seconds. BERT-Large had a F1 score of 0.700, but had the lowest Exact Match at 0.40 and had the longest inferencing time of 10 seconds, which is consistent with the evidence of its computational complexity. The confidence scores among models differed, with DistilBERT having a generally higher reliability at 0.710 than the other models. These results illustrate the trade-offs between accuracy, confidence, and computation efficiency across models.

IV. Discussion

a) Baseline Model

Predicted Answer	BERT Model	Exact Match	F1 Score	Confidence Score	Inference Time (seconds per query)
200	deepset/bert-base-cased-squad2 (baseline)	0	0.5	0.445	5.245
Bernard Olalia	deepset/bert-base-cased-squad2 (baseline)	1	1	0.163	4.091
the recent bombings in Beirut.	deepset/bert-base-cased-squad2 (baseline)	0	0.571	0.367	4.123
111	deepset/bert-base-cased-squad2 (baseline)	1	1	0.516	5.2
110	deepset/bert-base-cased-squad2 (baseline)	1	1	0.924	4.106
Philippine government is facing several challenges, including securing landing rights	deepset/bert-base-cased-squad2 (baseline)	0	0.462	0.007	4.119
land and sea routes	deepset/bert-base-cased-squad2 (baseline)	1	1	0.586	5.246
PHP150,000	deepset/bert-base-cased-squad2 (baseline)	1	1	0.949	4.095
Hezbollah.	deepset/bert-base-cased-squad2 (baseline)	0	0	0.978	4.095
no Filipinos were hurt	deepset/bert-base-cased-squad2 (baseline)	0	0.4	0.628	5.211

Fig. 23. Baseline Model Logs

The baseline BERT model showed a moderate performance, with an Exact Match of 0.50 and an F1 score of 0.693. The lower EM in particular speaks to the strictness of the EM metric; in a handful of cases minor differences in the phrasing of the wording or punctuation led to a non-matching EM of the predicted answer to the correct information, where simply changing “200” to “more than 200” or “Hezbollah.” to “Hezbollah” led to penalties, but the F1 score captured the significant overlap in tokens, offering a more particular measure of evaluation.

Confidence scores varied widely, ranging from as low as 0.007 to a high of 0.978, demonstrating that the model can even sometimes correctly predict answers even when the confidence is low, highlighting the necessity of considering both the predicted text as well as the probability score. Average inference times amounted to 4.55 seconds per question on CPU, which is reasonable for extractive Q&A, but evidence that even models that are moderate in size, like BERT-base, impose a noticeable computational cost even when they are run on non-GPU hardware.

BERT-base-cased-SQuAD2 is a good baseline, accurately capturing relevant information pertaining to the prompt while still demonstrating very particular drawbacks of relying on strictly an EM scoring and inference time effectiveness.

b) DistilBERT Model

BERT Model	Exact Match	F1 Score	Confidence Score	Inference Time (seconds per query)
distilbert/distilbert-base-cased-distilled-squad	1	1	0.709	1.34
distilbert/distilbert-base-cased-distilled-squad	0	0.444	0.368	1.377
distilbert/distilbert-base-cased-distilled-squad	1	1	0.503	1.357
distilbert/distilbert-base-cased-distilled-squad	1	1	0.833	1.346
distilbert/distilbert-base-cased-distilled-squad	1	1	0.588	1.347
distilbert/distilbert-base-cased-distilled-squad	1	1	1.148	1.348
distilbert/distilbert-base-cased-distilled-squad	1	1	0.871	2.157
distilbert/distilbert-base-cased-distilled-squad	1	1	0.979	1.654
distilbert/distilbert-base-cased-distilled-squad	0	0.4	0.492	1.333

Fig. 24. DistilBERT Model Logs

DistilBERT exemplifies the value of pretraining fused with knowledge distillation. The model's high Exact Match Score Average of 0.80 and F1 Score Average of 0.884 indicate that the model harnesses the knowledge of its pretrained weights for robust predictions, achieved with an average of 1.46 seconds for every question posed. While measuring the EM misses, we see that in some cases slight wording differences appeared in the calculation and again show us that the pretraining provided semantic comprehension for the answer, but could not derive a strict penalty if a metric was predetermined. Therefore, DistilBERT has revealed how both pretraining and knowledge distillation can improve both the speed and reliability of extractive Q&A.

c) BERT-Large Model

BERT Model	Exact Match	F1 Score	Confidence Score	Inference Time (seconds per query)
bert-large-uncased-whole-word-masking-finetuned-squad	0	0	0.177	10.228
bert-large-uncased-whole-word-masking-finetuned-squad	0	0.667	0.395	9.436
bert-large-uncased-whole-word-masking-finetuned-squad	0	0.667	0.301	9.846
bert-large-uncased-whole-word-masking-finetuned-squad	1	1	0.69	10.229
bert-large-uncased-whole-word-masking-finetuned-squad	1	1	0.641	10.236
bert-large-uncased-whole-word-masking-finetuned-squad	0	0.667	0.395	9.984
bert-large-uncased-whole-word-masking-finetuned-squad	0	0.6	0.6	9.265
bert-large-uncased-whole-word-masking-finetuned-squad	1	1	0.963	10.294
bert-large-uncased-whole-word-masking-finetuned-squad	1	1	0.895	10.268
bert-large-uncased-whole-word-masking-finetuned-squad	0	0.4	0.399	10.199

Fig. 25. BERT-Large Model Logs

BERT-Large demonstrates how the size of pretrained architectures affects performance. With 24 transformer blocks and 340 million parameters, BERT-Large's pretraining allows for a level of semantic reasoning which results in detailed answers, such as the name of "DMW Undersecretary Bernard Olalia." The tradeoff for this semantic richness is a lowered Exact Match of only 0.40 because strict EM scoring penalizes additional context, even if that context is correct. A F1 score average of 0.700 suggests that the token overlap is still reasonable. BERT-Large's broad confidence range and long-inference time of 10 seconds per question means that there is an important tradeoff to consider when comparing pre-trained model size with effectiveness and operation. BERT-Large shows that rich pretraining abilities offer nuanced understanding, while increasing compute and penalties in strict metrics.

d) RoBERTa Model

RoBERTa provides a great example of how advanced pretraining can lead to valuable performance at the model level. Through dynamic masking, significant training time on a larger training corpus, and strong pretraining, the model was able to generalize well, achieving an Exact Match score average of 0.70 and an F1 score average of 0.873. There will still be penalties in the EM metric for small variations from the gold standard answer, but the high F1 score average suggests that the model was able to use it pretrained semantic knowledge to accurately recall tokens. The confidence scores of the responses, which were between 0.287 and 1.261, also suggest that pretraining knowledge may add to the reliability of the answer. It is also observable that inference time averaged 2.8 seconds a question, which also speaks to efficiency with which pretraining occurs. RoBERTa has shown the capacity for improved pretraining to level semantic understanding and accuracy.

e) Visualizations

I.) Exact Match Score Average per Model

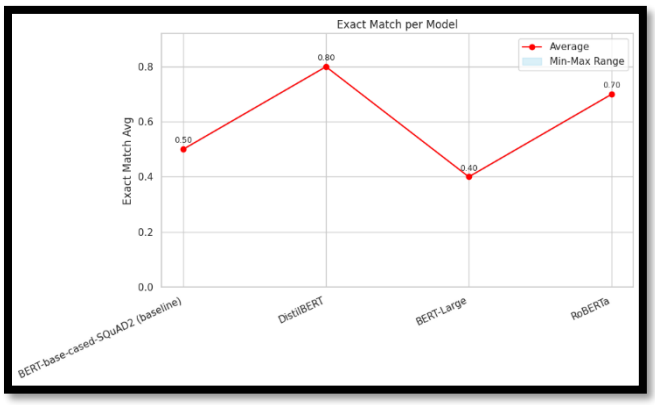


Fig. 26. Exact Match Score Average per Model Graph

The Exact Match average chart shows that the top score of 0.8 was from DistilBERT, which corresponds to predicting eight out of ten questions correctly. RoBERTa came in next at 0.7, indicating good performance but slightly below DistilBERT. BERT baseline had a decent score of 0.5. BERT-Large scored the lowest with a score of 0.4. This trend shows how smaller optimized models, like DistilBERT can beat out larger models at strict token-level matching, this may be attributed to shorter answer predictions and/or more effective pre-training overall.

II.) F1 Score Average per Model

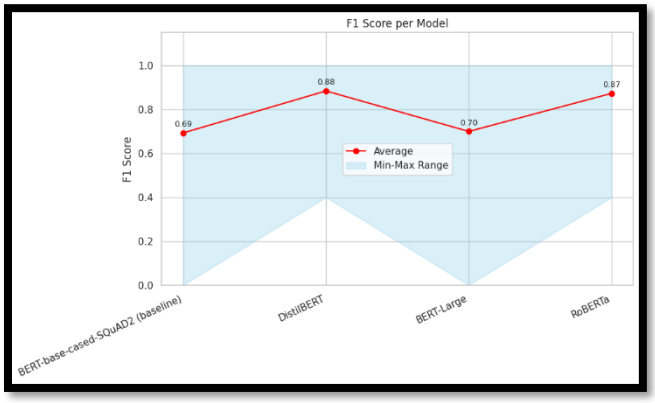


Fig. 27. F1 Score Average per Model Graph

The plot for the F1 score shows overlap between predicted tokens and tokens from the gold answer at the

token-level. Of the four models, DistilBERT tops the list scoring an average of 0.8844, meaning that most predicted answers captured almost all relevant tokens. Similarly high semantic accuracy is noted for RoBERTa, however it is marginally lower, scoring 0.8734. The baseline BERT model scored 0.6933 and BERT-Large model scored 0.7001. Even though BERT-Large's Exact Match score average was low, its F1 score suggests that there was a lot of overlap between the predicted and the correct answers. These differences show how F1 provides a much richer assessment of performance than EM alone.

III.) Confidence Score Average per Model

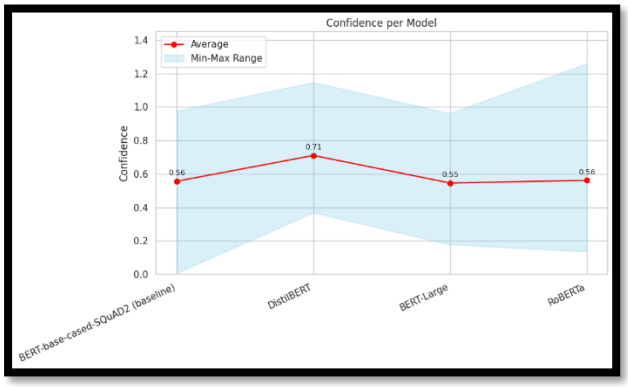


Fig. 28. Confidence Score Average per Model Graph

The confidence plot indicates that DistilBERT had the greatest average confidence of 0.7103, showing a solid degree of certainty with most of the predictions. Baseline BERT and RoBERTa both had similar average confidence values of 0.5563 and 0.5613 respectively, and BERT-Large had a slightly lower average of 0.5456. The min-max ranges show the variability in certainty, and DistilBERT displayed a higher minimum confidence than the baseline models, indicating that predictions are consistently more stable. These results emphasize the importance of pretraining and distilling a model for improving reliability in prediction.

IV.) Inference Time Average per Model

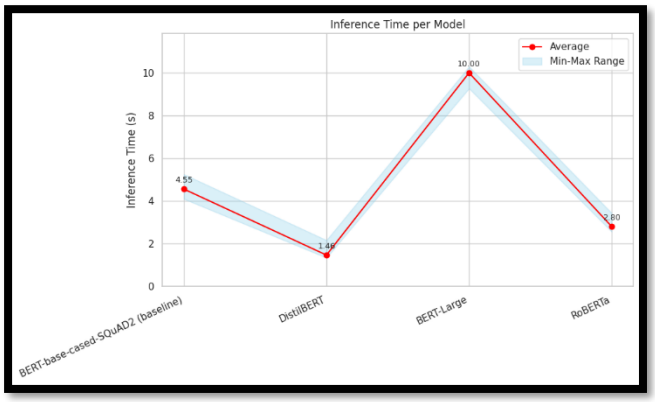


Fig. 29. Inference Time Average per Model Graph

The inference time plot demonstrates that DistilBERT is the most efficient, averaging 1.4598 seconds per question, followed by RoBERTa at 2.8008 seconds. The baseline BERT model required 4.5531 seconds, while BERT-Large was significantly slower at 9.9985 seconds due to its large number of parameters and transformer layers. These results show the trade-off between model size, complexity, and computational efficiency, with smaller or distilled models offering faster processing without sacrificing accuracy.

V.) Discussion Synthesis

The graphs demonstrate the trade-offs between accuracy, confidence, and efficiency for various pretrained Q&A models. DistilBERT produced the best Exact Match and F1 scores with a fast inference time and high confidence score, suggesting the success of the model distillation, as opposed to pretraining with the rest of the models. Although RoBERTa showed comparable accuracy scores to DistilBERT, its inference time was slightly longer, suggesting that optimizing strategies do have a performance effect. While BERT-Large is a larger representation of the BERT architecture, it produced lower Exact Match scores with much longer inference times, which shows that larger models do not necessarily produce better outputs for extractive Q&A. The original BERT baseline model mildly performed across all the metrics to create comparability. Findings suggest that the model, pretraining and fine-tuning is the best model to establish the trade-off situation for balancing semantic certainty, reliability, and computation efficiency.

V. Conclusion

This study evaluated four pretrained Q&A models, including BERT-base-cased-SQuAD2, DistilBERT, BERT-Large, and RoBERTa, on ten questions that required contextual information. DistilBERT exhibited the best overall combination of accuracy, confidence, and inference speed, highlighting the potential benefits of model distillation, especially for extractive Q&A tasks, to a more practical approach. RoBERTa achieved strong accuracy and token overlap, but was slightly slower. BERT-Large had the most capacity, but also had lower Exact Match scores and significantly longer inference times. The performance of the baseline BERT model is considered acceptable, but provided little more than a form of additional context from which improvements in the new models were observed. Model setup, specific pretrained model selections and proper evaluation metrics resulted in greater implications of predictive accuracy, as well as operating speed.

a) Challenges and Future Work

Exact Match scoring incorrectly penalizes answers that are correct but do not have the exact wording, potentially underestimating the model performance. Variability in confidence scores further suggests that having a numerical credential does not assure correctness. In addition, larger models, such as BERT-Large, require more computational

resources, and thus have limited applicability for practical usage in real-time environments.

Future work may address ensemble methods or hybrid approaches varying multiple pretrained models to improve robustness. Fine-tuning domain-specific corpora, advanced tokenization methods, and optimization techniques like quantization or pruning will improve accuracy and inference times. Lastly, expanding the question set more than 10 and utilizing more varied contexts may provide a better understanding of model generalization and semantic understanding.

VI. References

[1] Massed Compute, "What are the benefits of using pre-trained language models in machine learning?", Massed Compute, [Online]. Available: https://massedcompute.com/faq-answers/?question=What%20are%20the%20benefits%20of%20using%20pre-trained%20language%20models%20in%20machine%20learning? [Accessed: 29-Sep-2025].

[2] A. Farea, F. Emmert-Streib, "Pre-Trained Language Models and Their Applications," ScienceDirect, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197625009972 [Accessed: 29-Sep-2025].

[3] R. A. Gita-Carlos, "Chartered flights eyed for repatriation of over 200 OFWs in Lebanon," Philippine News Agency, Oct. 2, 2024. [Online]. Available: https://www.pna.gov.ph/articles/1234579 [Accessed: Sep. 29, 2025].

[4] Dataloop AI, “deepset/bert-base-cased-squad2,” Dataloop Library, 2025. [Online]. Available: https://dataloop.ai/library/model/deepset_bert-base-cased-squad2 [Accessed: 29-Sep-2025].

[5] V. Sanh, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper, lighter," Medium, 2019. [Online]. Available: https://medium.com/huggingface/distilbert-8cf3380435b5 [Accessed: 29-Sep-2025].

[6] Dataloop AI, "BERT Large Model (Uncased) Whole Word Masking," Dataloop AI, 2025. [Online]. Available: https://dataloop.ai/library/model/google-bert_bert-large-uncased-whole-word-masking-finetuned-squad/ [Accessed: 29-Sep-2025].

[7] Dataloop AI, “deepset/roberta-base-squad2," Dataloop AI, 2025. [Online]. Available: https://dataloop.ai/library/model/deepset_roberta-base-squad2/ [Accessed: 29-Sep-2025].